Experiment 7

Student Name: Lakshay Aggarwal
UID: 25MCI10047
Branch: MCA (AI/ML)
Section/Group: 25MAM-1(A)
Semester: 2-SEM
Date of Performance: 31/03/2026
Subject Name: Technical Training - 1
Subject Code: 25CAP-652

1. Aim of the Session:

Implementation of joins in PostgreSQL (inner join ,left join,right join, self join and cross join)

2. Software Requirements:

- PostgreSQL
- pgAdmin
- Windows Operating System

3. Objectives of the Session:

Apply joins to a real-world database schema (e.g., Students, Courses, Enrollments, Departments)

4. Theory:

A **JOIN** in SQL is used to combine rows from two or more tables based on a related column between them. Joins are essential in relational databases because data is often stored in multiple tables to reduce redundancy and improve organization.

Types of Joins Used in this Experiment:

1. **INNER JOIN:**
   Returns only the matching records from both tables.
2. **LEFT JOIN:**
   Returns all records from the left table and the matching records from the right table. If no match is found, NULL values are returned for the right table columns.
3. **RIGHT JOIN:**
   Returns all records from the right table and the matching records from the left table. If no match is found, NULL values are returned for the left table columns.
4. **SELF JOIN:**
   A table is joined with itself to represent relationships within the same table, such as mentor-student or manager-employee relationships.
5. **CROSS JOIN:**
   Returns the Cartesian product of two tables, i.e., all possible combinations of rows from both tables.

5. Practical / Experiment Steps:

Step 0: Creating tables and inserting records

Code:

```
create table student(
s_id int primary key,
s_name varchar(50),
dept_id int
);

create table department(
dept_id int primary key,
dept_name varchar(50)
);

create table course(
c_id int primary key,
c_name varchar(50)
);

create table enrollment(
s_id int,
c_id int,
primary key (s_id,c_id),
foreign key (s_id) references student(s_id),
foreign key (c_id) references course(c_id)
);

INSERT INTO department VALUES (1, 'CSE');
INSERT INTO department VALUES (2, 'IT');
INSERT INTO department VALUES (3, 'ECE');

INSERT INTO student VALUES (101, 'Lakshay', 1);
INSERT INTO student VALUES (102, 'neha', 2);
INSERT INTO student VALUES (103, 'tushar', 1);
INSERT INTO student VALUES (104, 'hardik', 3);
INSERT INTO student VALUES (105, 'kashish', 2);

INSERT INTO course VALUES (201, 'Oracle');
INSERT INTO course VALUES (202, 'SAP');
INSERT INTO course VALUES (203, 'IBM');
INSERT INTO course VALUES (204, 'Microsoft');
```

```
INSERT INTO enrollment VALUES (101, 201);
INSERT INTO enrollment VALUES (101, 202);
INSERT INTO enrollment VALUES (102, 203);
INSERT INTO enrollment VALUES (103, 201);
INSERT INTO enrollment VALUES (104, 204);
```

```
select * from student;
select * from department;
select * from course;
select * from enrollment;
```

Output:

	s_id [PK] integer	s_name character varying (50)	dept_id integer
1	101	Lakshay	1
2	102	neha	2
3	103	tushar	1
4	104	hardik	3
5	105	kashish	2

	c_id [PK] integer	c_name character varying (50)
1	201	Oracle
2	202	SAP
3	203	IBM
4	204	Microsoft

	dept_id [PK] integer	dept_name character varying (50)
1	1	CSE
2	2	IT
3	3	ECE

	s_id [PK] integer	c_id [PK] integer
1	101	201
2	101	202
3	102	203
4	103	201
5	104	204

Step 1: Write queries to list students with their enrolled courses (INNER JOIN).

Code:

```
SELECT s.s_id, s.s_name, c.c_name
FROM student s
INNER JOIN enrollment e ON s.s_id = e.s_id
INNER JOIN course c ON e.c_id = c.c_id;
```

Output:

	s_id integer	s_name character varying (50)	c_name character varying (50)
1	101	Lakshay	Oracle
2	101	Lakshay	SAP
3	102	neha	IBM
4	103	tushar	Oracle
5	104	hardik	Microsoft

Step 2Find students not enrolled in any course (LEFT JOIN).

Code:

```
SELECT s.s_id, s.s_name
FROM student s
LEFT JOIN enrollment e ON s.s_id = e.s_id
WHERE e.c_id IS NULL;
```

Output:

	s_id [PK] integer	s_name character varying (50)
1	105	kashish

Step 3: Display all courses with or without enrolled students (RIGHT JOIN).

Code:

```
SELECT s.s_name, c.c_name
FROM enrollment e
RIGHT JOIN course c ON e.c_id = c.c_id
LEFT JOIN student s ON e.s_id = s.s_id;
```

Output:

	s_name character varying (50)	c_name character varying (50)
1	Lakshay	Oracle
2	Lakshay	SAP
3	neha	IBM
4	tushar	Oracle
5	hardik	Microsoft

Step4: Show students with department info using SELF JOIN or multiple joins.

CODE:

```
SELECT s.s_id, s.s_name, d.dept_name
FROM student s
INNER JOIN department d ON s.dept_id = d.dept_id;
```

OUPUT:

	s_id integer	s_name character varying (50)	dept_name character varying (50)
1	101	Lakshay	CSE
2	102	neha	IT
3	103	tushar	CSE
4	104	hardik	ECE
5	105	kashish	IT

STEP5: Display all possible student-course combinations (CROSS JOIN). (Oracle, SAP, IBM, Microsoft)

CODE:

```
SELECT s.s_name, c.c_name
FROM student s
CROSS JOIN course c;
```


```
SELECT s.s_name, c.c_name
FROM student s
CROSS JOIN course c;
```


OUTPUT:

Showing rows: 1 to 20 Page No: 1

	s_name character varying (50)	c_name character varying (50)
1	Lakshay	Oracle
2	neha	Oracle
3	tushar	Oracle
4	hardik	Oracle
5	kashish	Oracle
6	Lakshay	SAP
7	neha	SAP
8	tushar	SAP
9	hardik	SAP
10	kashish	SAP
11	Lakshay	IBM
12	neha	IBM
13	tushar	IBM
14	hardik	IBM

6. Learning Outcomes (What I have learned):

- **Join Understanding:** I learned how to use different types of SQL joins to retrieve related data from multiple tables.
- **Practical Database Skills**: I understood how real-world schemas such as Students, Courses, Enrollments, and Departments are related in a relational database.
- **Data Analysis Ability**: I learned how to identify matching and non-matching records using INNER JOIN, LEFT JOIN, and RIGHT JOIN**.**
- **Advanced SQL Usage**: I understood the use of SELF JOIN for hierarchical relationships and CROSS JOIN for generating all possible combinations**.**
- **PostgreSQL Practice:** I gained hands-on experience in writing and executing SQL queries in PostgreSQL / pgAdmin.